

DAY 28 · ANNOTATION DAY 10

Reference Type & Primitive Type Dependency Injection

Injecting reference type variables the Spring 6.x way, and the four ways to inject primitive type values.



AGENDA

What we will cover today

- Part 1 — Reference type injection using Spring 6.x (Autowired moves into the source code class)
- Part 2 — Primitive type injection: four approaches (XML, Annotation + XML, Annotation, Java Configuration)

From Spring 5.x to Spring 6.x

In the previous class, we saw how to inject a reference type variable using the Java Configuration approach with `@Autowired`, using Spring 5.x dependencies.

Now let's look at the same example using Spring 6.x.

There is just one change: instead of writing `@Autowired` inside the bean (configuration class), we now write it inside the source code class itself.

Rocket.java

● Rocket.java

```
package com.day27;

import org.springframework.beans.factory.annotation.Autowired;

public class Rocket {

    Launcher launcher;
    B b;
    //Repair Center ;

    public Rocket() {
        System.out.println("Rocket constructor");
    }

    @Autowired
    public void setLauncher(Launcher launcher) {
        this.launcher = launcher;
    }

    @Autowired
    public void setB(B b) {
        this.b = b;
    }
}
```

Launcher.java

● Launcher.java

```
package com.day27;

public class Launcher {

    public Launcher() {
        System.out.println("Launcher Constructor ");
    }

    public void powerSupplied() {
        System.out.println("Rocket fuel & all setup done!");
    }
}
```

AutowireConfig.java

● AutowireConfig.java

```
package com.day27;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;

@Configuration
public class AutowireConfig {

    public AutowireConfig() {
        System.out.println("Java configuration class constructor ");
    }

    @Bean
    public Rocket rocket() {
        return new Rocket();
    }

    @Bean
    public Launcher launcher1() {
        return new Launcher();
    }

    @Bean
    @Primary
    public Launcher launcher2() {
```

LaunchCenter.java (Main class)

● LaunchCenter.java

```
public class LaunchCenter {  
    public static void main(String[] args) {  
  
        ApplicationContext context = new  
            AnnotationConfigApplicationContext(AutowireConfig.class);  
  
        Rocket bean = context.getBean("rocket", Rocket.class);  
        bean.ignite();  
    }  
}
```

Only one bean is marked @Primary for Launcher and one for B, so Spring knows exactly which bean to inject when setLauncher() and setB() run.

 PART 2

Injecting the Primitive Type

- Suppose we have a class where we want to store values.
- There are 4 approaches to assign values to variables — let's go through them one by one.

Four ways to inject primitive values

We will use the same example throughout — a cricket Player class with name, team, jersey number, matches, and runs.

1

XML Approach

Values are set directly inside application-context.xml using <property>.

2

Annotation + XML (combination)

@Value reads from a Properties bean created via <util:properties>, while XML still drives component scanning.

3

Annotation Approach

@PropertySource points to the properties file and @Value("\${...}") reads each key.

4

Java Configuration Approach

A @Configuration class reads values from the Environment object and builds the bean in a @Bean method.

Setter injection through application-context.xml

● application-context.xml

```
<beans>
  <bean id="player"
class="com.day28.Player">
    <property name="name" value="Ganguli"/>
    <property name="team" value="INDIA"/>
    <property name="jersyNo" value="99"/>
    <property name="matches" value="300"/>
    <property name="runs" value="10000"/>
  </bean>
</beans>
```

● Main.java

```
class Main {
    public static void main(String[] args) {
        ApplicationContext context =
            new
            ClassPathXmlApplicationContext(
                "com/day/application-context.xml");
        context.getBean("player",
            Player.class);
    }
}
```

Player.java

● Player.java

```
package com.day28;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class Player {

    @Value("#{playerInfo.name}")
    private String name;
    @Value("${playerInfo.team}")
    private String team;
    @Value("${playerInfo.jerseyNo}")
    private int jerseyNo;
    @Value("${playerInfo.matches}")
    private int matches;
    @Value("${playerInfo.runs}")
    private int runs;

    public void setName(String name) { this.name = name; }
    public void setTeam(String team) { this.team = team; }
    public void setJerseyNo(int jerseyNo) { this.jerseyNo = jerseyNo; }
    public void setMatches(int matches) { this.matches = matches; }
    public void setRuns(int runs) { this.runs = runs; }

    @Override
    public String toString() {
        return "Player [name=" + name + ", team=" + team +
            " jerseyNo=" + jerseyNo + " matches=" + matches +
            " runs=" + runs + "]";
    }
}
```

Properties file & XML configuration

● application.properties

```
name=Dhoni  
jersyNo=7  
matches=150  
runs=10000  
team=INDIA
```

● application-context.xml

```
<util:properties id="playerInfo"  
    location=  
        "classpath:com/day28/application.properties"/>  
  
<context:component-scan  
    base-package="com.day28"/>
```

As we are using both annotation and XML, the IoC container is created as a `ClassPathXmlApplicationContext`, which reads the XML configuration file.

But our class also has annotations, so `<context:component-scan>` is added to read the stereotype annotations as well.

Internally, `<context:component-scan>` also calls `<context:annotation-config>`, which registers all the `BeanPostProcessors` required to support the annotations inside the class.

Here, `<util:properties>` creates a `java.util.Properties` bean by reading the `application.properties` file and storing all its key-value pairs inside that bean.

Main.java

● Main.java

```
class Main {  
    public static void main(String[] args) {  
        ApplicationContext context =  
            new ClassPathXmlApplicationContext(  
                "com/day/application-context.xml");  
        context.getBean("player", Player.class);  
    }  
}
```

Player.java

● Player.java

```
package com.day28;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.PropertySource;
import org.springframework.stereotype.Component;

@Component
@PropertySource("classpath:com/day28/application.properties")
public class Player {

    @Value("${name}")
    private String name;
    @Value("${team}")
    private String team;
    @Value("${jersyNo}")
    private int jersyNo;
    @Value("${matches}")
    private int matches;
    @Value("${runs}")
    private int runs;

    public void setName(String name) { this.name = name; }
    public void setTeam(String team) { this.team = team; }
    public void setJersyNo(int jersyNo) { this.jersyNo = jersyNo; }
    public void setMatches(int matches) { this.matches = matches; }
    public void setRuns(int runs) { this.runs = runs; }

    @Override
    public String toString() {
```

Properties file & Main class

● application.properties

```
name=Virat  
jersyNo=18  
matches=200  
runs=12000  
team=INDIA
```

● Main.java

```
public class Main {  
    public static void main(String[] args) {  
        ApplicationContext context = new  
            AnnotationConfigApplicationContext(  
                "com.day28");  
  
        Player bean = context.getBean(  
            "player", Player.class);  
        System.out.println(bean);  
    }  
}
```

Player.java & application.properties

Player.java

```
package com.day28;

public class Player {

    private String name;
    private String team;
    private int jersyNo;
    private int matches;
    private int runs;

    public void setName(String name) { this.name = name; }
    public void setTeam(String team) { this.team = team; }
    public void setJersyNo(int jersyNo) {
        this.jersyNo = jersyNo;
    }
    public void setMatches(int matches) {
        this.matches = matches;
    }
    public void setRuns(int runs) { this.runs = runs; }

    @Override
    public String toString() {
        return "Player [name=" + name + ", team=" + team +
            ", jersyNo=" + jersyNo + ", matches=" + matches +
            ", runs=" + runs + "]\n";
    }
}
```

application.properties

```
name=Rohit
jersyNo=45
matches=200
runs=11000
team=INDIA
```


JavaConfiguration.java

● JavaConfiguration.java

```
package com.day28;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.PropertySource;
import org.springframework.context.annotation.PropertySources;
import org.springframework.core.env.Environment;

@Configuration
@PropertySources({
    @PropertySource("classpath:com/day28/application1.properties"),
    @PropertySource("classpath:com/day28/application.properties")}
public class JavaConfiguration {

    @Autowired
    Environment environment;

    @Bean
    public Player player() {
        Player player = new Player();
        player.setName(environment.getProperty("name"));
        player.setJerseyNo(Integer.parseInt(
            environment.getProperty("jerseyNo")));
        player.setMatches(Integer.parseInt(
            environment.getProperty("matches")));
        player.setRuns(Integer.parseInt(
            environment.getProperty("runs")));
        player.setTeam(environment.getProperty("team"));
        return player;
    }
}
```

How the Environment object works

When you write the `@PropertySource` annotation in the Java Configuration approach, the IoC container creates a separate object inside itself, known as the Environment.

Inside the Environment, property sources are stored as a chain. Here we wrote one `@PropertySource` — `@PropertySource("classpath:com/day28/application.properties")` — so one property source exists in the Environment.

If we write two, they form a chain: if the data is not found in one file, Spring looks in the next one. The syntax is `@PropertySources({ @PropertySource("...application1.properties"), @PropertySource("...application.properties") })`.

In this way, we can write multiple property sources. All the key-value pairs from the properties files end up inside the `PropertySource` in the Environment object, inside the IoC container. So, to read any key and value, we go through the Environment object.

Main.java

● Main.java

```
public class Main {  
    public static void main(String[] args) {  
        ApplicationContext context = new  
            AnnotationConfigApplicationContext(  
                JavaConfiguration.class);  
  
        Player bean = context.getBean(  
            "player", Player.class);  
        System.out.println(bean);  
    }  
}
```

In both the Annotation approach and the Java Configuration approach, `@PropertySource` works exactly the same way, as explained above.

Putting it all together

Note: In both the Annotation approach and the Java Configuration approach, @PropertySource works in the same way as explained earlier.



Injecting a primitive type

- application.properties
- @PropertySource()
- @Value



Injecting a reference type

- @Autowired
- @Primary — used when multiple beans of the same type are available